SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA1409 - COMPILER DESIGN

LEX Programs 1–10

Name: E.Ragul

Register Number: 192571032

Department: B.Tech Computer Science and BioScience

SIMATS Engineering, Chennai

Experiment No. 1

Experiment Name: Count Characters, Words, and Lines using LEX

Aim

To write a LEX program to count the number of characters, words, and lines in a given C source file.

Procedure

1. Create a C source file named sample.c.
2. Write the LEX program and save it as count_lines.l.
3. Open the command prompt in the folder containing both files.
4. Compile the LEX program using flex count_lines.l and gcc lex.yy.c and a.exe sample.c.
5. Display the number of characters, words, and lines.

Program: count_lines.l

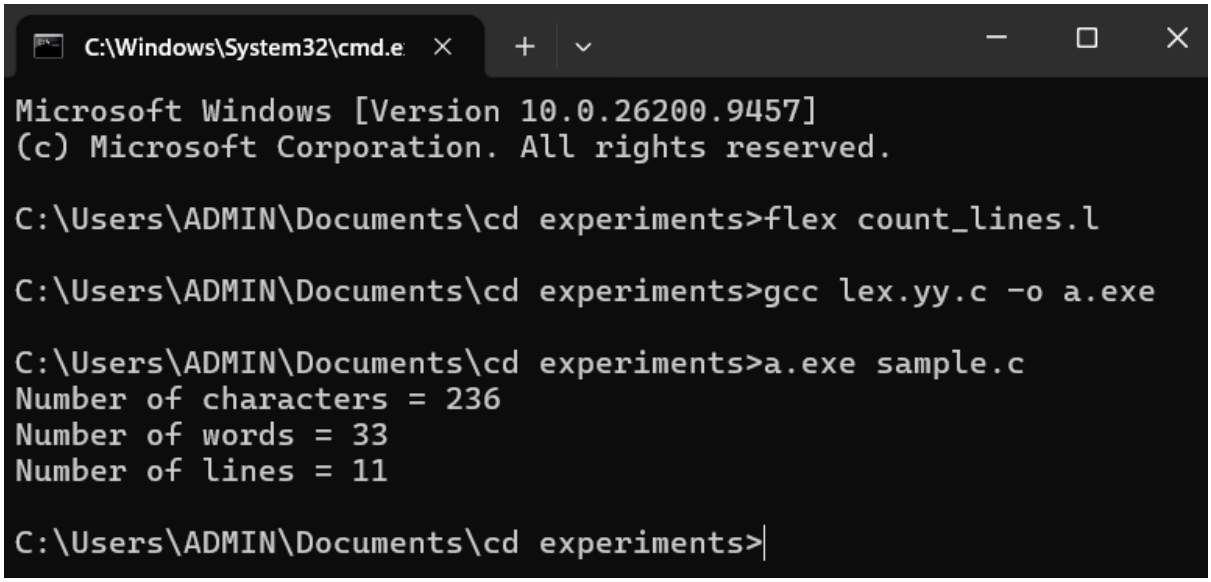
```
%{
#include <stdio.h>
int nchar, nword, nline;
%}
%%
\n          { nline++; nchar++; }
[^ \t\n]+   { nword++; nchar += yyleng; }
.           { nchar++; }
%%
int yywrap(void)
{
   return 1;
}
int main(int argc, char *argv[])
   yyin = fopen(argv[1], "r");
   yylex();
   printf("Number of characters = %d\n", nchar);
```

```
    printf("Number of words = %d\n", nword);
    printf("Number of lines = %d\n", nline);
    fclose(yyin);
    return 0;
}
```

Input Source Program: sample.c

```
#include <stdio.h>
int main()
{
    int number1, number2, sum;
    printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2);
    sum = number1 + number2;
    printf("%d + %d = %d", number1, number2, sum);
    return 0;
}
```

Output



Result

Thus, the LEX program was written and executed to count the number of characters, words, and lines in a given C source file.

Experiment No. 2

Experiment Name: Print Constants using LEX

Aim

To write a LEX program to print all constants in a given C source program file.

Procedure

1. Create a C source file named sample.c.
2. Write the LEX program and save it as countconstants.l.
3. Open the command prompt in the folder containing both files.
4. Compile the LEX program using flex countconstants.l.
5. Compile the generated file using gcc lex.yy.c.
6. Execute the program using a.exe.
7. Enter the source file name and display all constants.

Program: countconstants.l

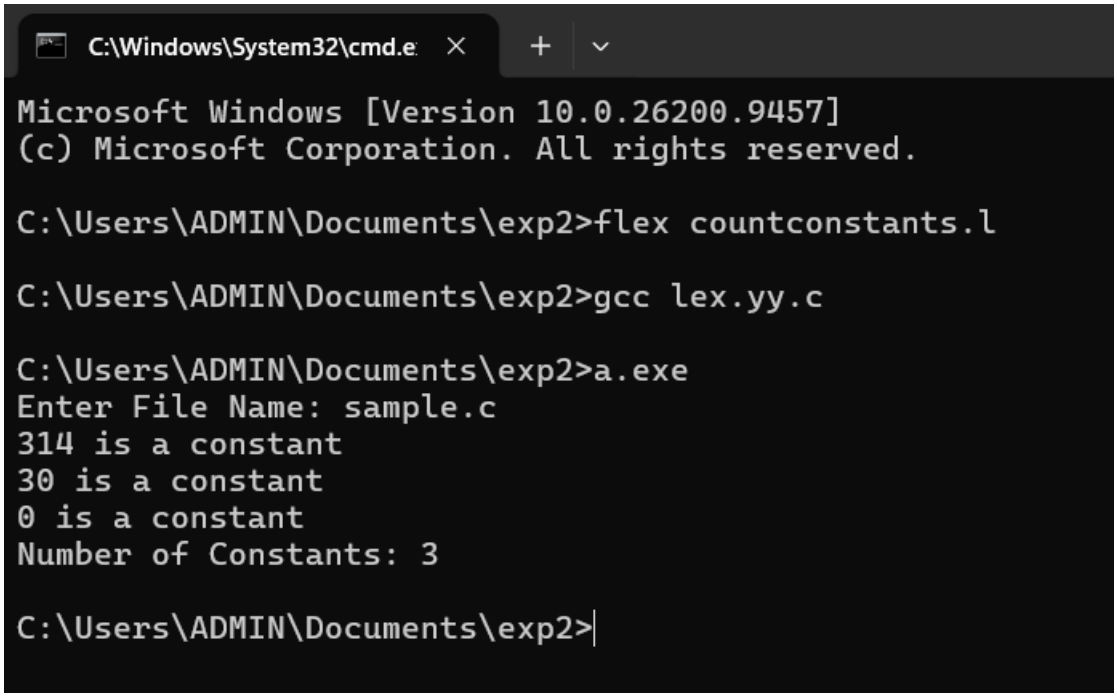
```
%{
#include <stdio.h>
int cons = 0;
%}
digit [0-9]
%%
{digit}+ {
   cons++;
   printf("%s is a constant\n", yytext);
}
.|\n { }
%%
int yywrap(void)
{
   return 1;
}
```

```
int main(void)
{
   FILE *f;
   char file[100];
   printf("Enter File Name: ");
   scanf("%99s", file);
   f = fopen(file, "r");
   if (f == NULL)
   {
      printf("File not found!\n");
      return 1;
   }
   yyin = f;
   yylex();
   printf("Number of Constants: %d\n", cons);
   fclose(f);
   return 0;
}
```

Input Source Program: sample.c

```
#define P 314
#include <stdio.h>
#include <conio.h>
void main()
{
   int a, b, c = 30;
   printf("hello");
}
```

Output



Result

Thus, the LEX program was written and executed successfully to print and count all constants in a given C source program file.

Experiment No. 3

Experiment Name: Count Macros and Header Files using LEX

Aim

To write a LEX program to count the number of macros defined and header files included in a C program.

Procedure

1. Create a C source file named sample.c.
2. Write the LEX program and save it as count_macro.l.
3. Compile the program using flex count_macro.l and gcc lex.yy.c and sample.c.
4. Display the number of macros and header files.

Program: count_macro.l

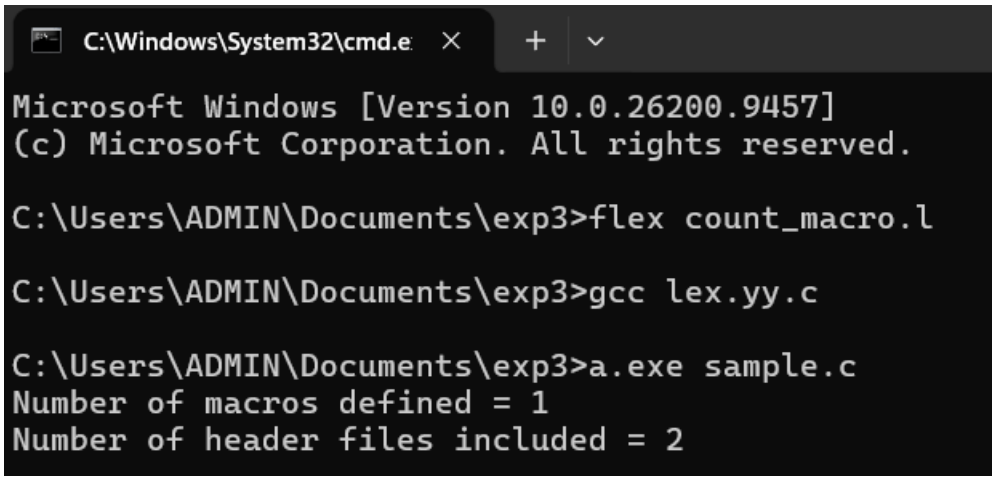
```
%{
#include <stdio.h>
int nmacro = 0, nheader = 0;
%}
^#define    { nmacro++; }
^#include   { nheader++; }
%%
int yywrap(void)
{
   return 1;
}
int main(int argc, char *argv[])
   if (argc < 2)
      printf("Usage: a.exe sample.c\n");
      return 1;
   yyin = fopen(argv[1], "r");
   if (yyin == NULL)
      printf("File not found!\n");
```

```
        return 1;
    }
    yylex();
    printf("Number of macros defined = %d\n", nmacro);
    printf("Number of header files included = %d\n", nheader);
    fclose(yyin);
    return 0;
}
```

Input Source Program: sample.c

```
#define PI 3.14
#include <stdio.h>
#include <conio.h>
void main()
{
    int a, b, c = 30;
    printf("hello");
}
```

Output



Result

Thus, the LEX program was written and executed to count the macros and header files in a C program.

Experiment No. 4

Experiment Name: Print HTML Tags using LEX

Aim

To write a LEX program to print all HTML tags in an input file.

Procedure

1. Create an HTML file named sample.html.
2. Write the LEX program and save it as html.l.
3. Compile the program using flex html.l.
4. Compile the generated file using gcc lex.yy.c.
5. Execute the program using a.exe.
6. Enter the HTML file name and display all tags.

Program: html.l

```
%{
#include <stdio.h>
int tags = 0;
%}
%%
"<"[^>]*> {
   tags++;
   printf("%s\n", yytext);
}
.|\n { }
%%
int yywrap(void)
{
   return 1;
}
int main(void)
{
```

```
    FILE *f;
    char file[100];

    printf("Enter File Name: ");
    scanf("%99s", file);

    f = fopen(file, "r");

    if (f == NULL)
    {
        printf("File not found!\n");
        return 1;
    }

    yyin = f;
    yylex();

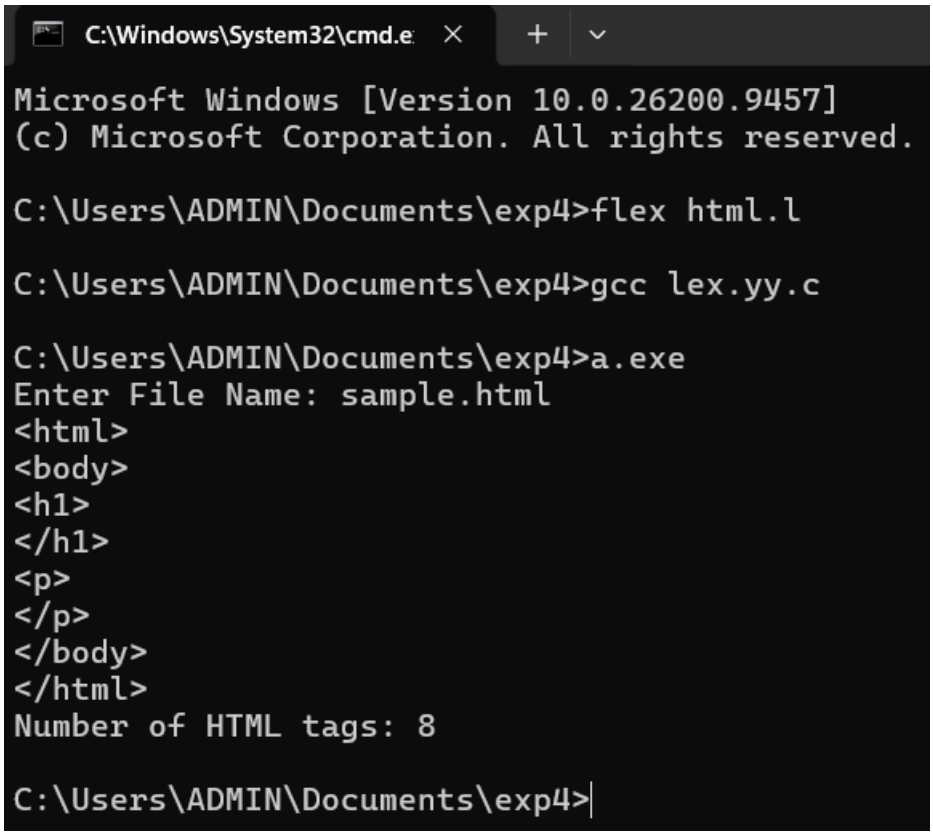
    printf("Number of HTML tags: %d\n", tags);

    fclose(f);
    return 0;
}
```

Input Source Program: sample.html

```
<html>
<body>
<h1>My First Heading</h1>
<p>My first paragraph.</p>
</body>
</html>
```

Output



Result

Thus, the LEX program was written and executed to print and count all HTML tags in an input file.

Experiment No. 5

Experiment Name: Add Line Numbers using LEX

Aim

To write a LEX program to add line numbers to a given C program and display it on the screen.

Procedure

1. Create a C source file named sample.c.
2. Write the LEX program and save it as addlinenos.l.
3. Compile the program using flex addlinenos.l and sample.c and a.exe sample.c.
4. Display the C program with line numbers.

Program: addlinenos.l

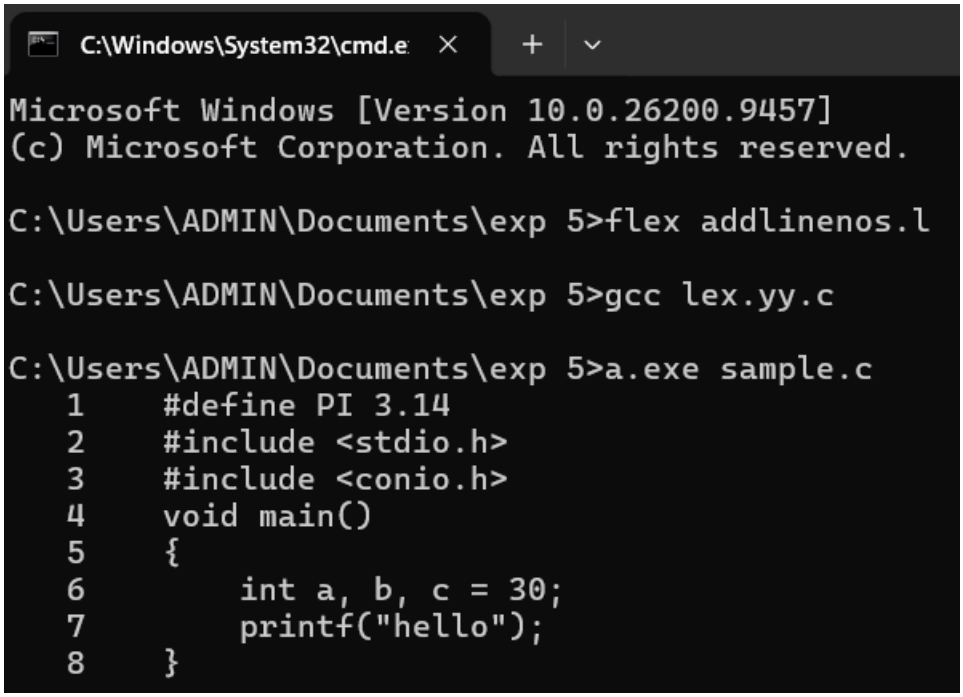
```
%{
#include <stdio.h>
int line = 1;
%}
%%
.*\n    { printf("%4d\t%s", line++, yytext); }
.+      { printf("%4d\t%s\n", line++, yytext); }
%%
int yywrap(void)
int main(int argc, char *argv[])
{
   if (argc < 2)
   {
      printf("Usage: a.exe sample.c\n")
   yyin = fopen(argv[1], "r");
   if (yyin == NULL)
   {
      printf("File not found!\n");
```

```
        return 1;
    }
    yylex();
    fclose(yyin);
    return 0;
}
```

Input Source Program: sample.c

```
#define PI 3.14
#include <stdio.h>
#include <conio.h>
void main(){
    int a, b, c = 30;
    printf("hello");
}
```

Output



Result

Thus, the LEX program was written and executed to add line numbers to a given C source program and display it on the screen.

Experiment No. 6

Experiment Name: Remove and Count Comments using LEX

Aim

To write a LEX program to count comment lines in a C program, remove the comments, and write the remaining program into another file.

Procedure

1. Create an input file named input.c.
2. Write the LEX program and save it as comment.l.
3. Compile using flex comment.l.
4. Compile the generated file using gcc lex.yy.c.
5. Execute using a.exe input.c output.c.
6. Display the number of comments removed.
7. Check the comment-free program in output.c.

Program: comment.l

```
%{
#include <stdio.h>
#include <stdlib.h>
int com = 0;
%}
%s COMMENT
%%
"/*"            { BEGIN(COMMENT); }
<COMMENT>"*/"   { BEGIN(INITIAL); com++; }
<COMMENT>\n     { com++; }
<COMMENT>.      { }
"//"[^\n]*      { com++; }
.|\n            { fprintf(yyout, "%s", yytext); }
%%
```

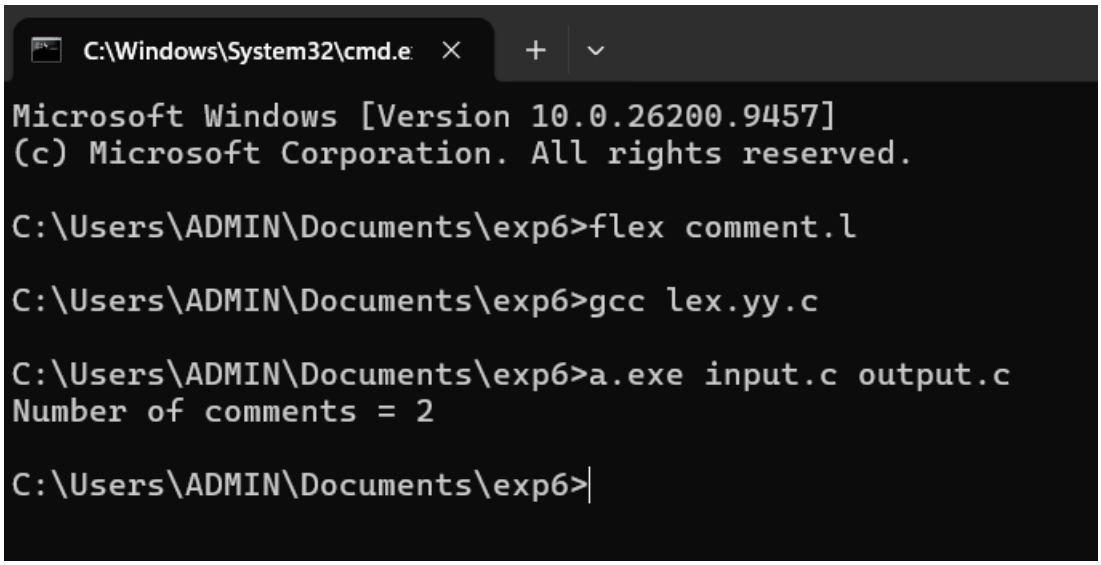
```
int yywrap(void)
{
   return 1;
}
int main(int argc, char *argv[])
{
   if (argc != 3)
   {
      printf("Usage: a.exe input.c output.c\n");
      return 1;
   }
   yyin = fopen(argv[1], "r");
   yyout = fopen(argv[2], "w");

   if (yyin == NULL || yyout == NULL)
   {
      printf("File opening error!\n");
      return 1;
   }
   yylex();
   printf("Number of comments = %d\n", com);
   fclose(yyin);
   fclose(yyout);
   return 0;
}
```

Input Source File: input.c

```
#include <stdio.h>
int main()
{
   int a, b, c; /* variable declaration */
   printf("Enter two numbers");
   scanf("%d %d", &a, &b);
   c = a + b; // adding two numbers
   printf("Sum is %d", c);
   return 0;
}
```

Output



Result

Thus, the LEX program was written and executed to count and remove comments from a C source file.

Experiment No. 7

Experiment Name: Identify Capital Words using LEX

Aim

To write a LEX program to identify capital words in a given input string.

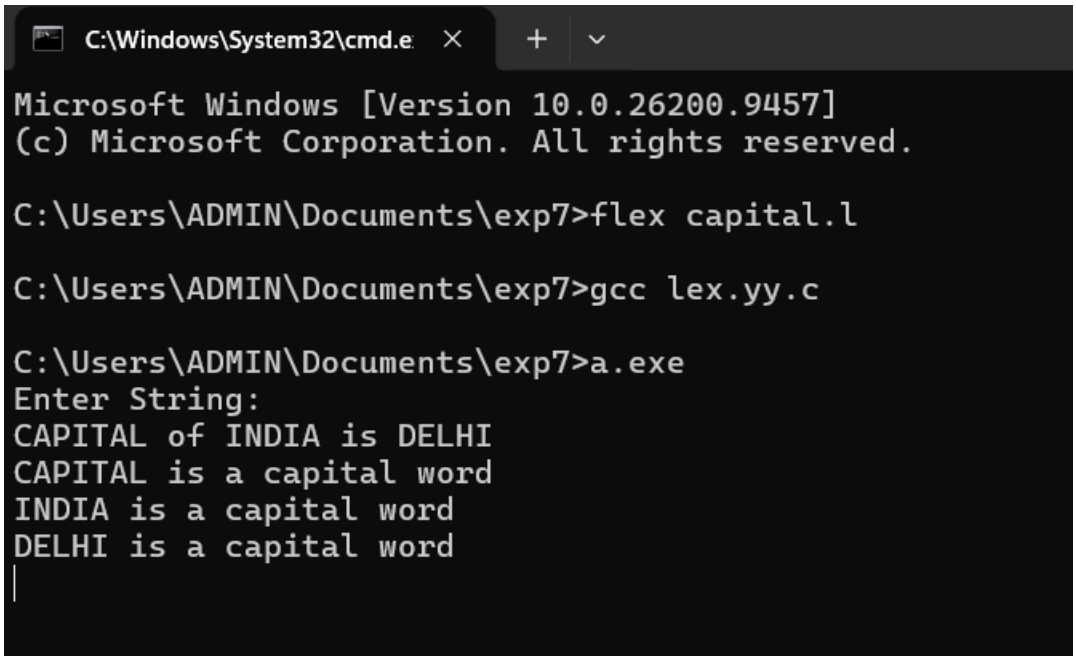
Procedure

1. Write the LEX program and save it as capital.l.
2. Compile using flex capital.l.
3. Compile the generated file using gcc lex.yy.c.
4. Execute the program using a.exe.
5. Enter a string containing capital words.
6. Display the identified capital words.

Program: capital.l

```
%{
#include <stdio.h>
%}
%%
[A-Z]+  { printf("%s is a capital word\n", yytext); }
.|\n    { }
%%
int main(void)
{
   printf("Enter String:\n");
   yylex();
   return 0;
}
int yywrap(void)
{
   return 1;
}
```

Output



Result

Thus, the LEX program was written and executed to identify capital words in the given input string.

Experiment No. 8

Experiment Name: Validate Email Address using LEX

Aim

To write a LEX program to check whether a given email address is valid or invalid.

Procedure

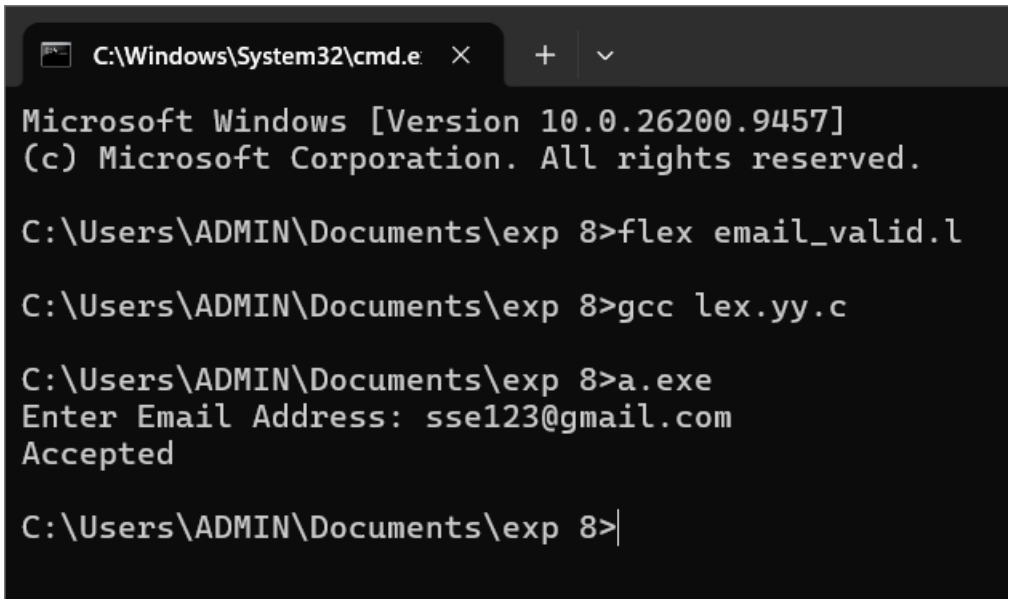
1. Write the LEX program and save it as email_valid.l.
2. Compile using flex email_valid.l.
3. Compile the generated file using gcc lex.yy.c.
4. Execute the program using a.exe.
5. Enter an email address.
6. Display whether the email address is accepted or not accepted.

Program: email_valid.l

```
%{
#include <stdio.h>
%}
%%
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.(com|in) {
   printf("Accepted\n");
}
.+ {
   printf("Not Accepted\n");
}
\n { }
%%
int main(void){
   printf("Enter Email Address:\n");
   yylex();
   return 0;
}
```

```
int yywrap(void)
{
   return 1;
}
```

Output



Result

Thus, the LEX program was written and executed to check whether the given email address is valid or invalid.

Experiment No. 9

Experiment Name: Convert abc to ABC using LEX

Aim

To write a LEX program to convert the substring abc into ABC in a given input string.

Procedure

1. Write the LEX program and save it as substring.l.
2. Compile the program using flex substring.l.
3. Compile the generated file using gcc lex.yy.c.
4. Execute the program using a.exe.
5. Enter a string containing the substring abc.
6. Display the string after conversion.

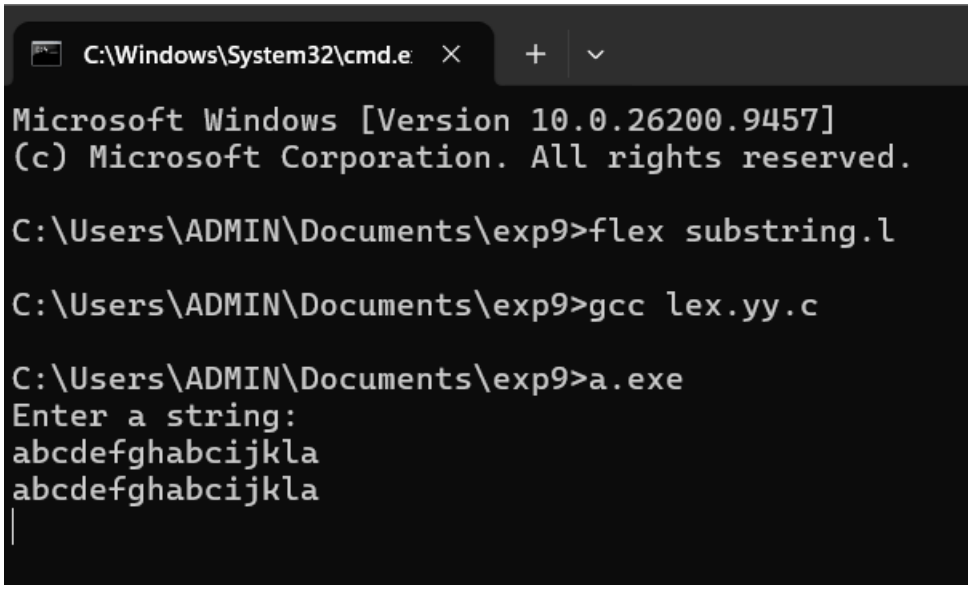
Program: substring.l

```
%{
#include <stdio.h>
#include <string.h>
%}

%%
[a-zA-Z]+ {
   int i;
   for (i = 0; i < yyleng; i++)
   {
      if (yytext[i] == 'a' &&
          yytext[i+1] == 'b' &&
          yytext[i+2] == 'c')
      {
          yytext[i] = 'A';
          yytext[i+1] = 'B';
          yytext[i+2] = 'C';
```

```
            i += 2;
        }
    }
    printf("%s", yytext);
}
[ \t]+  { printf("%s", yytext); }
\n      { printf("\n"); }
.       { printf("%s", yytext); }
%%
int main(void)
    yylex();
    return 0;
int yywrap(void)
{
    return 1;
}
```

Output



Result

Thus, the LEX program was written and executed to convert the substring abc into ABC in the given input string.

Experiment No. 10

Experiment Name: Validate Mobile Number using LEX

Aim

To write a LEX program to check whether a given mobile number is valid or invalid.

Procedure

1. Write the LEX program and save it as mobile.l.
2. Compile using flex mobile.l.
3. Compile the generated file using gcc lex.yy.c.
4. Execute the program using a.exe.
5. Enter a mobile number.
6. Display whether the mobile number is valid or invalid.

Program: mobile.l

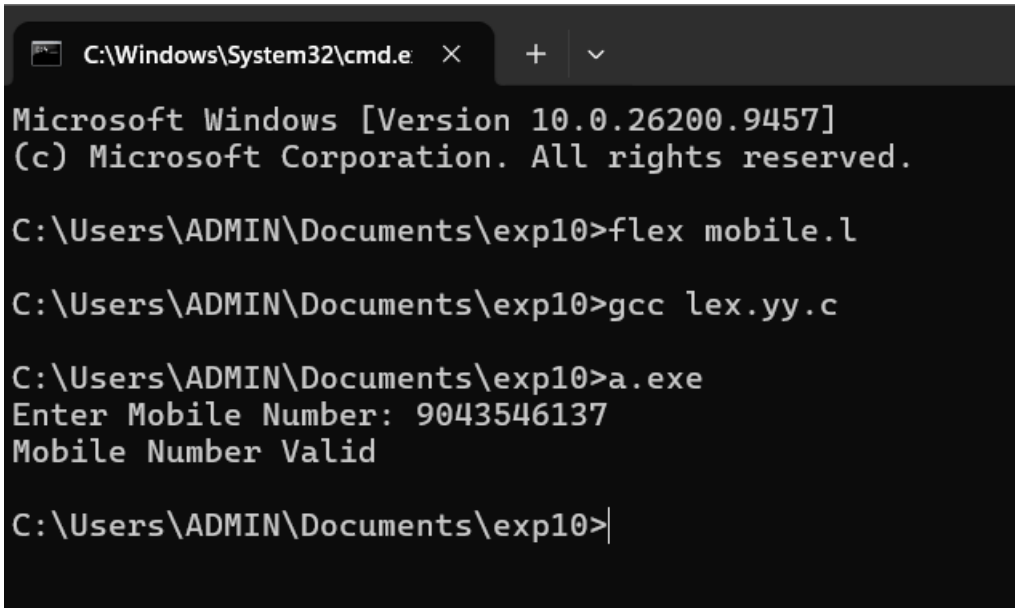
```
%{
#include <stdio.h>
%}

%%
[1-9][0-9]{9}   { printf("\nMobile Number Valid\n"); }
.+              { printf("\nMobile Number Invalid\n"); }
\n              { }
%%

int main(void)
{
   printf("Enter Mobile Number: ");
   yylex();
   return 0;
}
```

```
int yywrap(void)
{
   return 1;
}
```

Output



Result

Thus, the LEX program was written and executed to check whether the given mobile number is valid or invalid.